□]633;E;for f in $(find . -type f -name "*.md" ! -name "syllabus.md" ! -name "book_pim_core_developer_manual.md" | sort);9973c574-14b1-4098-811a-73beeba064a5□□]633;C□

01.md

📝 3-Sentence Summary

Pimcore is an open-source **digital experience platform** built on Symfony that centralizes and manages structured and unstructured data. It combines **PIM (Product Information Management), MDM (Master Data Management), DAM (Digital Asset Management), CMS/Headless CMS, and eCommerce frameworks** into one system, so businesses don't need separate tools. Its core building blocks—**Data Objects, Classes, Assets, Documents, and an extensible Admin UI**—let developers model complex data, manage content, and expose it via APIs.

🔑 Main Problems Pimcore Solves

- **Data chaos** → unifies scattered product, asset, and content data.
- **Content silos** → allows single-source publishing across websites, apps, marketplaces, and print.
- **Integration pain** → API-first and Symfony-based, so it plugs into modern stacks.
- **Scalability** → handles millions of products, files, or pages efficiently.

🌍 Main Use Cases

- **PIM**: Manage product catalogs, specs, attributes, multi-language data.

- **MDM**: Single source of truth for enterprise data.
- **DAM**: Organize, transform, and deliver digital assets (images, videos, docs).
- **CMS / Headless CMS**: Manage websites or deliver structured content to apps via APIs.
- **eCommerce**: Extend with product management, carts, pricing, personalization.

⚙️ Core Components

- **Data Objects** → structured business entities (Products, Categories, Customers).
- **Classes** → schemas that define the structure of Data Objects.
- **Assets** → files (images, videos, PDFs) with transformations (e.g., thumbnails).
- **Documents** → content pages (CMS, landing pages, snippets).
- **Admin UI** → ExtJS-based backend for managing everything.
- **Symfony integration** → full access to Symfony's controllers, services, events, and bundles.

🎯 Easy Analogy

Think of Pimcore like a **"digital warehouse + factory"**:

- **Warehouse shelves (Data Objects & Assets)** store your products, images, and content.
- **Blueprints (Classes)** define how things are structured.
- **Factory machines (Admin UI + APIs)** let you organize, enrich, and distribute data anywhere. Instead of running five different apps (PIM, DAM, CMS, etc.), Pimcore is like having one big warehouse with all the right sections under one roof.

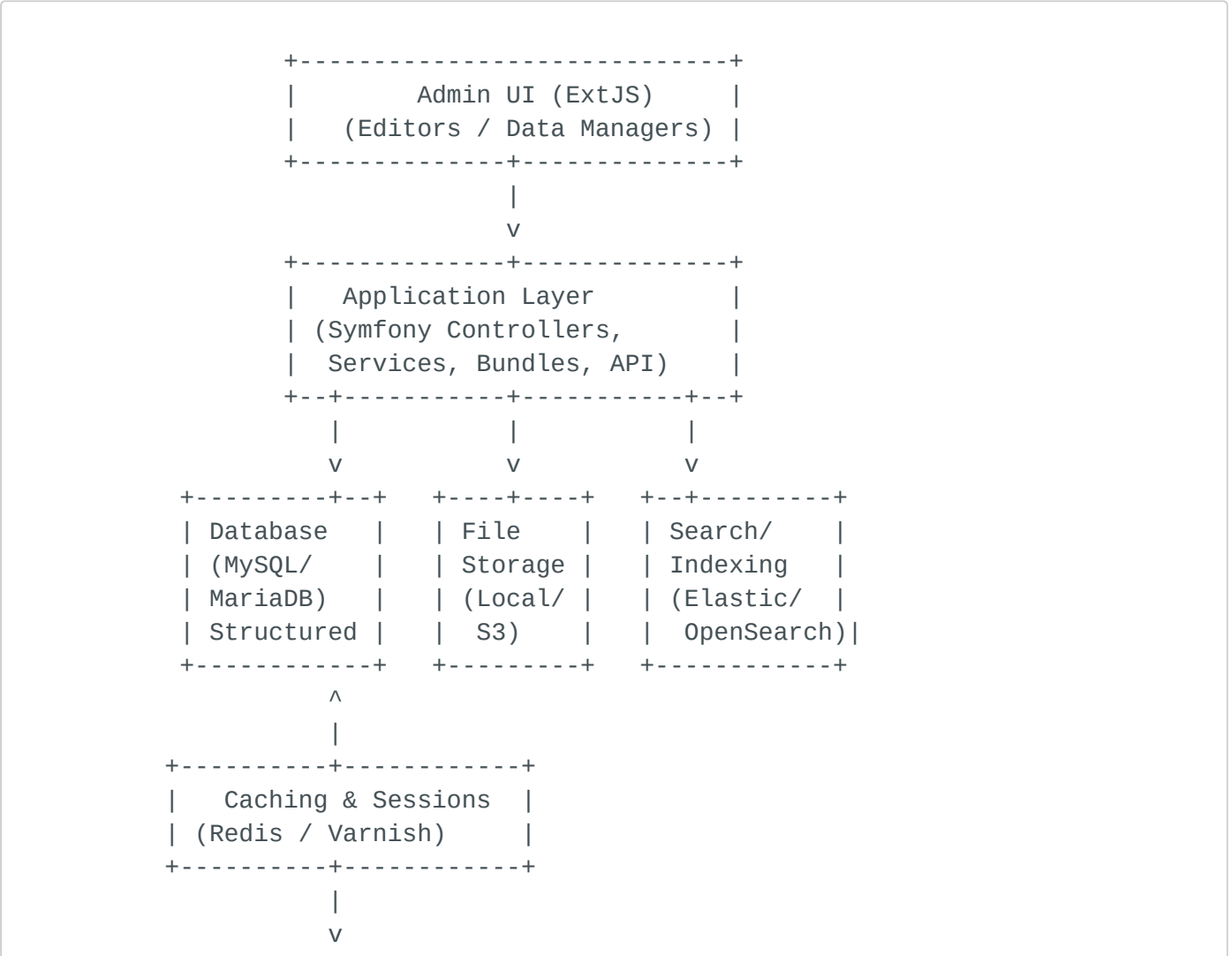
02.md

🏗️ Pimcore Architecture (Typical Setup)

Pimcore is built on **Symfony PHP framework** and has a modular architecture that connects different layers:

- **Admin UI** → ExtJS-powered backend for editors/admins.
- **Application Layer** → Symfony controllers, services, bundles, events.
- **Persistence** → MySQL/MariaDB for structured data (Data Objects, Classes, Documents metadata).
- **File Storage** → local FS, NFS, or cloud (S3, Azure Blob, etc.) for assets (images, videos, PDFs).
- **Search & Indexing** → Elasticsearch / OpenSearch (optional, but critical for large catalogs).
- **Caching & Sessions** → Redis or Varnish for fast delivery.
- **APIs** → REST, GraphQL (via DataHub), custom Symfony endpoints.
- **Frontend** → Symfony templates (Twig), headless delivery (Next.js, Vue, React, Angular, etc.), or classic CMS templates.

🔍 ASCII Diagram



```
+----------+------------+
|   APIs & Frontends    |
| (REST, GraphQL,       |
|  Headless JS Apps,    |
|  Websites w/ Twig)    |
+-----------------------+
```

⚙️ Typical Services Needed in Production

1. **Web Server / App Runtime**
   - PHP-FPM (>=8.1)
   - Nginx or Apache
2. **Database**
   - MySQL / MariaDB (primary data store)
3. **Cache & Sessions**
   - Redis (highly recommended)
   - Optional: Varnish for full-page caching
4. **File Storage**
   - Local file system for small setups
   - NFS / S3 / Azure Blob for scalable asset storage
5. **Search / Indexing**
   - Elasticsearch / OpenSearch for large catalogs & fast queries
6. **Message Queue (optional, for async tasks)**
   - RabbitMQ or Symfony Messenger with Redis transport
7. **Reverse Proxy / CDN**
   - Nginx, Cloudflare, Akamai, or Fastly for global distribution
8. **Monitoring & Logging**

- Monolog, ELK stack, or Grafana + Prometheus

👉 In short: Pimcore sits at the **core application layer**, MySQL stores structured data, S3 (or local disk) stores assets, Elasticsearch makes searching fast, Redis/Varnish makes it snappy, and your frontend(s) consume everything via API or CMS templates.

03.md

Nice — let's get you a **copy-pasteable, step-by-step** Pimcore local dev setup on Linux (Docker + `docker compose` + Composer). I'll give exact terminal commands, the few edits you'll want to make, common pitfalls and fixes, and how to open the Admin UI the first time.

> **Quick checklist** (you'll run these in order): install Docker + compose → create project with Pimcore image → edit `docker-compose.yaml` user line → `docker compose up -d` → run `pimcore-install` → open `http://localhost/admin`.

---

1) Prerequisites (install Docker + Docker Compose plugin)

If Docker isn't installed yet, install Docker Engine and the Compose plugin. On Ubuntu/Debian the commands look like:

```
# (1) Basic packages
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release

# (2) Add Docker GPG key and repo (official Docker docs)
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-
plugin

# allow your user to run docker (log out / in afterwards or run `newgrp
docker`)
sudo usermod -aG docker $USER
newgrp docker   # or log out and log back in
```

Verify:

```
docker run --rm hello-world
docker compose version
```

(If you prefer another distro, follow Docker's official install pages — these commands are the Ubuntu flow.) (Docker Documentation)

---

2) Create a new Pimcore project (demo or skeleton) using the official Pimcore image

Use the Pimcore Docker image to run Composer so you don't need PHP locally.

From the folder where you want the project:

```
mkdir ~/pimcore && cd ~/pimcore

# demo (has sample data/UI examples)
docker run -u $(id -u):$(id -g) --rm -v $(pwd):/var/www/html \
  pimcore/pimcore:php8.3-latest composer create-project pimcore/demo my-
project

# OR skeleton (empty starter for production apps)
docker run -u $(id -u):$(id -g) --rm -v $(pwd):/var/www/html \
  pimcore/pimcore:php8.3-latest composer create-project pimcore/skeleton my-
project
```

Notes:

- `-u $(id -u):$(id -g)` ensures files on your host are created with your UID/GID to avoid permission pain.
- The Pimcore team publishes PHP-versioned images (e.g. `php8.3-latest`). Use the tag that matches your target PHP version. (docs.pimcore.com)

---

3) Inspect & adapt the `docker-compose.yaml`

Enter the project and open the repo's `docker-compose.yaml`:

```
cd my-project
ls -la
# print your uid:gid (for docker-compose user lines)
echo $(id -u):$(id -g)
```

The skeleton/demo includes a `docker-compose.yaml`. The docs recommend *uncommenting* the `user:` lines and replacing with your `uid:gid` if needed — this prevents file permission headaches. Then start the stack:

```
# start services in background
docker compose up -d

# see containers and mapped ports
docker compose ps
```

If you ever need a shell inside the PHP container:

```
docker compose exec php bash
```

(If `docker compose` isn't available, you may have an older `docker-compose` binary — install the plugin or follow Docker docs.) (docs.pimcore.com)

---

4) Run the Pimcore installer (automated)

Once the containers are up, initialize Pimcore and the DB. Example automated command (non-interactive DEV credentials):

```
docker compose exec php vendor/bin/pimcore-install \
  --admin-username=admin --admin-password=admin \
  --mysql-username=pimcore --mysql-password=pimcore --mysql-database=pimcore
\
  --mysql-host-socket=db --no-interaction
```

This runs the installer inside the `php` container and creates the DB/schema and admin user. For demo installs this can take longer (assets, indexes, sample data). You can change `--admin-username`/`--admin-password` to something secure for non-dev use. (docs.pimcore.com)

---

5) Open the site and Admin UI

- Frontend: `http://localhost` (or the host/port shown by `docker compose ps`).
- Admin UI: `http://localhost/admin` — log in with the admin username/password you passed to `pimcore-install` (e.g., `admin`/`admin` in the example). (docs.pimcore.com)

If the Admin UI doesn't load, check `docker compose ps` for port mappings and `docker compose logs php` for errors.

---

6) Useful commands while developing

```
# follow logs
docker compose logs -f php
docker compose logs -f db

# open a shell in php container
docker compose exec php bash

# run symfony console inside container
docker compose exec php bin/console cache:clear
docker compose exec php bin/console pimcore:system:requirements:check
```

```
# stop & remove containers + volumes (clean)
docker compose down -v --remove-orphans
```

---

7) Common pitfalls & how to fix them

1. `docker` commands require sudo / permission errors

- If `docker run` fails: `sudo systemctl enable --now docker` and/or add your user to the `docker` group (`sudo usermod -aG docker $USER`) then re-login. (Docker Documentation)

2. `docker compose` command not found

- Install Docker Compose plugin (`docker-compose-plugin`) or use the standalone binary per Docker docs. `docker compose version` verifies. (Docker Documentation)

3. File permission issues (created files owned by root / can't write cache)

- Edit `docker-compose.yaml` `user:` lines to your `UID:GID` (docs recommend this), or fix permissions on host:

  ```
  sudo chown -R $(id -u):$(id -g) .
  ```

  or inside container:

  ```
  docker compose exec php bash -lc "chown -R www-data:www-data var public vendor"
  ```

  The skeleton project's `docker-compose.yaml` already contains commented `user:` lines — uncomment & fill them. (docs.pimcore.com)

4. Database connection errors (SQLSTATE[HY000] [2002])

- Ensure DB container is running (`docker compose ps`) and `.env` / `APP_ENV` / database DSN points to the service name `db` (that's the host used inside the docker network). Check `docker compose logs db` for MySQL errors. (Server Fault)

5. Admin page missing / 404 after install

- Ensure your web server points at `my-project/public` (or that docker mapped the port to `public`), run `composer install` inside the project if you used composer locally, and check file permissions and `.htaccess` (Apache) or nginx configuration. See logs. (StackOverflow threads often point to missing composer install or wrong document root.) (Stack Overflow)

6. Elasticsearch / Search backend

- Demo projects may assume a search backend. If you enable the backend, ensure the corresponding service (Elasticsearch/OpenSearch) is present in `docker-compose.yml` and configured; the docs mention adjusting supervisor config to build the index. If you don't need search locally, you can disable the search backend until later. (docs.pimcore.com)

---

8) Extra tips & next steps

- Pimcore version requirements: recent Pimcore releases require modern PHP (Pimcore 2025+ moved PHP minimum to 8.3 in 2025 release notes) — align your image tag (php8.3) with that requirement. Run the system requirements check: `docker compose exec php bin/console pimcore:system:requirements:check`. (docs.pimcore.com)
- If you want the admin UI to be faster, use Redis for cache (the skeleton/docker-compose often includes a `redis` service — enable it).
- If you want to do frontend development (assets), the skeleton may include a `node` service — run `yarn install` and `yarn dev` either on host or in that container.

---

9) Short troubleshooting cheatsheet

- `docker compose ps` → check containers
- `docker compose logs -f php` → check PHP/Pimcore errors
- `docker compose exec php vendor/bin/pimcore-install --help` → see installer options
- `docker compose exec php bin/console cache:clear` → clear cache
- `docker compose down -v` → destroy volumes and start fresh

(If something fails, paste the output of `docker compose ps` and `docker compose logs php` and I'll help debug.)

Links / References

- Pimcore — Docker-based installation (official docs). (docs.pimcore.com)
- Pimcore — Advanced Installation (installer CLI args example). (docs.pimcore.com)
- Pimcore skeleton & demo packages (composer create-project instructions). (GitHub)
- Docker Engine & Docker Compose plugin installation docs. (Docker Documentation)

If you want, I can:

- paste a **ready-to-use** `docker-compose.yaml` (trimmed for local dev) you can drop into `my-project/` and `docker compose up -d` immediately, **or**
- run through the exact commands tailored to your Linux distro (Ubuntu/Debian/Fedora), or
- give the minimal commands to create the project without Docker (composer way).

Which one do you want next?

04.md

🧱 Core Building Blocks

1. Classes

- **What it is:** Schema (blueprint) that defines fields and structure for Data Objects.
- **When to use:** Whenever you need a structured entity type (e.g., Product, Category, Customer).

- **Example:**
  - `Product` class with fields: `name` (Text), `sku` (Input), `price` (Numeric), `images` (Asset relation).
  - Every product object you create will follow this schema.

---

2. Data Objects

- **What it is:** Actual instances (records) created from Classes. They are the *content/data* stored.
- **When to use:** For storing actual entries like individual products, categories, blog posts, etc.
- **Example:**
  - `Product` object = `"iPhone 16 Pro"` with SKU `IP16P-256`, price `₹120,000`.
  - Belongs to `Product` class.

---

3. Object Bricks

- **What it is:** Reusable sets of fields you can attach to different Classes dynamically.
- **When to use:** For optional or type-specific attributes that not every object has.
- **Example:**
  - `ElectronicsSpecs` brick with fields `batteryLife`, `screenSize`.
  - `ClothingSpecs` brick with `fabric`, `sizeGuide`.
  - A `Product` can have zero, one, or multiple Object Bricks depending on category.

---

4. Field Collections

- **What it is:** Groups of fields you can repeat multiple times within one object.
- **When to use:** When you need repeatable sets of structured data.
- **Example:**
  - In `Product`, you add a `Specification` field collection with fields `label` and `value`.
  - Product `"iPhone"` might have specs like:
    - (`label=Battery`, `value=24h`)
    - (`label=Weight`, `value=180g`)

---

5. Localized Fields

- **What it is:** Special container for fields that support multiple languages.
- **When to use:** If you need multi-language/global-ready content.
- **Example:**
  - In `Product`, fields like `name` and `description` stored inside Localized Fields.
  - For `"iPhone"` → `name[en]=iPhone 16`, `name[de]=iPhone 16`, `name[fr]=iPhone 16`.

---

6. Relations

- **What it is:** Links between objects, assets, or documents.
- **When to use:** When entities need to reference each other.
- **Types:**
  - **Object Relation**: Product → Category.
  - **Asset Relation**: Product → Image(s).
  - **Many-to-Many Relation**: Product → multiple Tags.
- **Example:**

- `Product` (`iPhone 16`) relates to:
  - Category `Smartphones` (object relation).
  - Assets: `iphone16-front.jpg`, `iphone16-back.jpg` (asset relations).
  - Tags: `5G`, `Apple`, `Premium` (many-to-many).

🛠️ Tiny Walkthrough Example

Imagine modeling **Products with Variants**:

1. Create a **Class**: `Product` → fields: `sku`, `name`, `price`.
2. Add **Localized Fields** for `name` and `description`.
3. Add **Object Relation** to `Category`.
4. Add **Asset Relation** for images.
5. Add an **Object Brick** `ElectronicsSpecs` → `battery`, `screenSize`.
6. Add a **Field Collection** `Technical Specs` for flexible attribute lists.
7. Create **Data Objects**:
   - `Product`: `"iPhone 16"` → has images + `ElectronicsSpecs`.
   - `Product`: `"iPhone 16 Pro"` → variant with higher price + extra specs.

👉 Together: **Classes define the structure**, **Data Objects are the actual records**, **Object Bricks/Field Collections/Localized Fields make it flexible**, and **Relations tie everything together**.

05.md

Create a Product class — two ways: Admin UI (click-by-click) and

programmatically (PHP + how to create product instances)

I'll show:

1. Exact click-by-click steps in the Admin UI (safe, recommended).
2. Two programmatic options: (A) recommended workflow — export/import class JSON + CLI, and (B) example PHP code to create **Product** data objects (instances) and set `images` (asset relation) and `categories` (object relation). I'll also list the common pitfalls and commands you'll need. Sources: Pimcore docs for Object Classes / Data Types / Working-with-PHP-API / Deployment Tools. (docs.pimcore.com)

---

1) Admin UI — click-by-click (recommended)

(Assumes you can log into Pimcore Admin at `http://<your-host>/admin`)

1. Log in → open **Settings** (gear icon) → **Data Objects** → **Classes**. (docs.pimcore.com)
2. Click **Add** (top right) → choose **Class** → enter **Name**: `Product` → click **Create**.
3. While editing the `Product` class, you'll see the layout editor: the left column has layout components (Panel, Tab Panel) and data components (Input, Textarea, Numeric, Relations, …). (docs.pimcore.com)
4. Add a main container panel (right-click `Base` → **Add Layout Component** → **Panel**) — this will be your main `Base` panel (title it `Base` or `Product data`).
5. Inside that panel, **Add data component** → **Text** → **Input**:
   - `Name` (field name): `sku`
   - `Title`: `SKU`
   - (Optional) check **Unique** and/or **mandatory** if you want SKU uniqueness.
6. Add another **Input**:

- `Name`: `name`
- `Title`: `Name`

7. Add **Number** → **Numeric**:
   - `Name`: `price`
   - `Title`: `Price`
   - You can set min/max, default, or unit handling later.
8. Add **Text** → **Textarea** or **WYSIWYG**:
   - `Name`: `description`
   - `Title`: `Description`
9. Add **Relations** → **Many-To-Many Relation** (for images):
   - `Name`: `images`
   - `Title`: `Images`
   - In the config set `types` → allow `asset` and subtypes `image` (or use `imagegallery`/`imageadvanced` types depending on need). Optionally enable inline upload. (docs.pimcore.com)
10. Add **Relations** → **Many-To-Many Object Relation** (for categories):
    - `Name`: `categories`
    - `Title`: `Categories`
    - In the config set **Allowed Classes** → choose `Category` (you must create a `Category` class first or allow any object class). (docs.pimcore.com)
11. (Optional) Configure how fields are visible in grid views (tick “visible in grid view” for sku/name/price).
12. Click **Save** (top-right). Pimcore will persist the class definition; the UI generates the class meta and stores the definition. (docs.pimcore.com)
13. If you want the generated PHP class and config files committed to code, **Export** (Classes list → select Product → Export JSON) or use deployment commands described below. (See **Deployment / Import** section.) (docs.pimcore.com)

---

2) Programmatic approaches

A — Recommended production/dev workflow (safe + reproducible)

1. Build the class in Admin UI exactly as you want. Export the class definition to JSON (Classes → select → **Export**). This gives you a canonical class JSON that Pimcore understands. (docs.pimcore.com)
2. Commit that JSON to your repo (e.g. `config/pimcore/classes/Product.json`), and on target instances import it with Pimcore console commands:

```
# import the class JSON once on the target system
php bin/console pimcore:definition:import:class /path/to/Product.json

# then rebuild classes / DB structures
php bin/console pimcore:deployment:classes-rebuild --create-classes

# clear cache
php bin/console cache:clear
```

Those commands are the canonical way to move class definitions between environments. (Good for CI/CD & reproducibility.) (docs.pimcore.com)

> Why this is recommended: class JSON is exactly what Pimcore uses internally — it avoids subtle API/namespace differences between Pimcore releases and gives you versionable class definitions.

B — Programmatic: create DataObjects (instances) in PHP (concrete example)

Assume you already created the `Product` class (via Admin UI or imported JSON + `classes-rebuild`) so the generated PHP model `Pimcore\Model\DataObject\Product` exists. Here's a practical PHP example that:

- creates or finds a parent folder `/products`
- creates a Product object with `name`, `sku`, `price`, `description`

- attaches assets to `images`
- attaches object references to `categories`
- saves the object

```php
<?php
// scripts/create_product.php   (run inside your Pimcore project root)
use Pimcore\Model\DataObject;
use Pimcore\Model\Asset;
use Pimcore\Model\Element\Service as ElementService;

// bootstrap Symfony / Pimcore if running standalone script
require __DIR__ . '/../vendor/autoload.php';
$kernel = new \App\Kernel('dev', true);
$kernel->boot();
$container = $kernel->getContainer();
\Pimcore::collectGarbage(); // optional housekeeping

// 1) ensure parent folder exists (create /products if missing)
$parent = DataObject\Folder::getByPath('/products');
if(!$parent) {
    $parent = new DataObject\Folder();
    $parent->setKey(ElementService::getValidKey('products', 'object'));
    $parent->setParentId(1); // id 1 is typically the root; adjust as needed
    $parent->setPath('/products');
    $parent->setName('Products');
    $parent->save();
}

// 2) create new Product object (assumes class Product exists)
$product = new DataObject\Product();
$product->setKey(ElementService::getValidKey('iPhone-16-pro', 'object'));
$product->setParentId($parent->getId());
$product->setName('iPhone 16 Pro');

// set your fields (use the field names defined in the class)
$product->setSku('IP16P-256');
$product->setPrice(120000);
$product->setDescription('Flagship phone — example description.');

// 3) attach images (assumes assets with these IDs exist)
$imageIds = [123, 124]; // replace with real asset IDs
$assets = [];
foreach ($imageIds as $id) {
    $asset = Asset::getById($id);
    if ($asset) {
        $assets[] = $asset;
    }
}
$product->setImages($assets); // Many-to-Many Asset Relation expects an
array of Asset objects

// 4) attach categories (assumes Category objects exist)
$categoryIds = [10, 11]; // replace with real object IDs for Category
objects
$cats = [];
```

```
foreach ($categoryIds as $cid) {
    $c = DataObject\Category::getById($cid);
    if ($c) {
        $cats[] = $c;
    }
}
$product->setCategories($cats); // Many-to-Many Object Relation expects
array of DataObject instances

// 5) save
$product->save(["versionNote" => "Created via script"]);

echo "Saved product with ID: " . $product->getId() . PHP_EOL;
```

Notes & sources: the generated model `DataObject\Product` and setters come from the class definition; see Pimcore's PHP API docs for object CRUD and the asset examples. (docs.pimcore.com)

Quick checklist & common pitfalls (so you don't get stuck)

- **"Class not found"** when using `DataObject\Product`: you created the class in the UI but didn't rebuild/generate classes. Run:

  ```
  php bin/console pimcore:deployment:classes-rebuild --create-classes
  composer dump-autoload --optimize
  php bin/console cache:clear
  ```

  (This generates the PHP classes under `var/classes` / `config/pimcore/classes`.) (docs.pimcore.com)

- **Setter names**: generated setters follow your field `name` exactly (camelCase). If your field name is `sku`, the setter is `setSku()`. If you rename fields, stored data can be lost — be careful. (docs.pimcore.com)

- **Parent folder / Parent ID**: pick a correct `parentId` (or use `Folder::getByPath('/products')`) — otherwise the object might be created in an unexpected place.

- **Assets / Objects not found**: `Asset::getById($id)` and `DataObject\Category::getById($id)` must return instances; otherwise the

setter gets `null` and may error.

- **Many vs Single relations**:
  - Single image fields (Image) expect an `Asset\Image` instance (not an array).
  - Many-to-many asset relation expects an **array of Asset objects**. Check your class field type. (docs.pimcore.com)

- **DB schema changes**: class changes may alter DB tables. Keep migrations in mind when moving between environments. The `classes-rebuild` command handles DB structure updates. (docs.pimcore.com)

- **Localization**: if you defined `name`/`description` inside a `Localized Fields` container, you must set locale-aware values (use generated methods or localized setters/listings). See the docs if you need i18n. (docs.pimcore.com)

Small cheatsheet of commands you'll use

```
# import class JSON (one-off)
php bin/console pimcore:definition:import:class /path/to/Product.json

# rebuild classes & DB structures
php bin/console pimcore:deployment:classes-rebuild --create-classes

# clear cache
php bin/console cache:clear

# run a PHP script from project root (example)
php bin/console d:f:run -- app:your-script    # or run a bootstrap script
directly via php scripts/create_product.php if bootstrapped
```

(Use the exact environment flags `--env=prod` if running in production context.) (docs.pimcore.com)

If you'd like, I can:

- produce a ready-to-import **Product class JSON** you can drop into `config/pimcore/classes` (I'll craft it from the fields you gave), **or**

- give a full example of a class-created-by-PHP script (the `ClassDefinition` object approach) — but note that the JSON-export/import workflow is more robust across Pimcore versions.

Which one do you want next?

06.md

Got it 👍 Let's break this down systematically. Pimcore manages **content** with three major building blocks:

- **Data Objects** → structured data (like `Product`, `Category`, etc.)
- **Assets** → raw files (images, PDFs, videos, etc.)
- **Documents** → pages, emails, snippets (content delivery and CMS layer)

Here we'll focus on **Assets** and **Documents**, their roles, and practical steps for media-heavy projects.

---

1. Assets in Pimcore

What they are

- The **Assets tree** in the admin UI is a hierarchical file system for all uploaded files.
- Types: Images, Videos, Documents (PDF, Word, etc.), Archives, Any binary.
- Each Asset is a Pimcore element (with ID, metadata, versions, permissions).
- Assets can be related to Data Objects (via `Image`, `Many-to-Many Asset Relation`, etc.).

Upload / manage

- Go to **Admin UI** → **Assets tab**.
- Create folders for organization (`/products`, `/banners`, `/videos`).
- Upload via **Drag & Drop** or **Upload button**.

- Metadata: Right-click asset → **Edit** → add custom metadata fields (e.g., license, photographer, alt text).
- Versioning: Pimcore stores previous versions of assets automatically.
- Permissions: Asset tree supports role-based access (who can upload/edit/delete).

Best practices for large libraries

- **Folder structure**: use logical hierarchies (e.g., `/products/electronics/phones`). Avoid dumping everything in `/`.
- **Metadata**: define required metadata fields (license, usage rights).
- **CDN / Reverse proxy**: serve assets via CDN for performance.
- **Image Thumbnails**: don't serve original huge images directly; always use thumbnails.
- **Asset Inheritance**: leverage relations to avoid duplication (e.g., share one asset across many products).
- **Archival**: use Pimcore's asset archive or external object storage (S3, MinIO, etc.) if assets are huge.

---

2. Documents in Pimcore

What they are

- **Documents** = pages, snippets, and emails built with Pimcore's CMS.
- Stored in the **Documents tree** (`/en/products/iphone`).
- Three main types:
  - **Page** → full CMS pages, can have editable areas (wysiwyg, image, object relations).
  - **Snippet** → reusable content block (like a footer, banner).
  - **Email** → templates for sending via Pimcore's mail system.

Usage

- Build page structures (e.g., `/home`, `/about`, `/products`).
- Insert editables (e.g., `wysiwyg`, `image`, `link`) in Twig templates.
- Manage translations with document versions for each language (`/en`, `/de`).
- Permissions: role-based access at folder/page level.

---

3. Image Transformations (Thumbnails)

What

- Pimcore **Thumbnail configurations** define transformations: resize, crop, frame, watermark, format conversion.
- They can be configured in **Admin UI → Settings → Thumbnails**, or programmatically.

Admin UI workflow

1. Go to **Settings → Thumbnails**.
2. Create a new thumbnail configuration, e.g. `product_preview`.
3. Add transformations:
    - Resize to 800×600
    - Crop center
    - Set output format (JPEG, WebP)
4. Save.
5. Use it in Twig / PHP:

    ```
    <img src="{{ asset.thumbnail('product_preview') }}">
    ```

    or in PHP:

```php
$thumb = $asset->getThumbnail('product_preview');
echo $thumb->getPath();
```

---

4. Programmatic Example — Create Thumbnail Variant

Here's how to define and apply a **thumbnail config** in PHP:

```php
use Pimcore\Model\Asset\Image;
use Pimcore\Model\Asset\Image\Thumbnail\Config as ThumbnailConfig;

// Load an image asset
$image = Image::getById(123); // replace with actual asset ID

// Define a thumbnail config programmatically
$thumbConfig = new ThumbnailConfig();
$thumbConfig->addItem('resize', [
    'width' => 400,
    'height' => 300,
    'forceResize' => true
]);
$thumbConfig->setFormat('webp');  // output format
$thumbConfig->setQuality(85);

// Generate the thumbnail (temporary, not stored in settings)
$thumbPath = $image->getThumbnail($thumbConfig)->getPath();

echo "Generated thumb path: " . $thumbPath;
```

👉 This creates an **on-the-fly thumbnail** without adding a permanent definition in the Admin UI. If you want reusable configs, define them in **Settings → Thumbnails**, then just call by name.

---

5. Serving Large Media Libraries

- **Never serve originals directly** → always use thumbnails.
- Use **different thumbnails for web vs. admin** (e.g., small 100×100 grid preview vs. 1200×800 web display).
- Use **WebP/AVIF** when possible for smaller file size.

- Set up **cache headers** and/or CDN (CloudFront, Akamai, Fastly, etc.).
- For huge files (videos, CAD models): consider storing them in S3 with Pimcore's S3 Asset Store plugin.
- For search & browsing: enable **Elasticsearch/Opensearch indexing** for assets.

✅ So:

- **Assets** → manage files (upload, metadata, thumbnails).
- **Documents** → build CMS pages/snippets/emails.
- **Image Thumbnails** → define transformations (UI or code) to serve optimized variants.

Would you like me to also create a **ready-to-import thumbnail config JSON** (so you can just drop it into Pimcore and test), or do you prefer sticking with the programmatic method?

07.md

Great question — Pimcore is **100% Symfony under the hood** 🐘, so all the best practices of Symfony apply. Pimcore adds its own layer (Admin UI, Data Objects, Assets, Documents), but you organize your code exactly as you would in a Symfony app.

📂 Typical Pimcore + Symfony Project Structure

```
project-root/
├─ bundles/                # Custom Pimcore bundles (like Symfony bundles)
├─ config/
│   ├─ packages/           # Symfony & Pimcore package configs
│   ├─ routes/             # YAML or PHP route configs
│   └─ services.yaml       # Your service definitions
├─ src/
│   ├─ Controller/         # Symfony controllers
│   ├─ Repository/         # Doctrine repositories (custom queries)
│   ├─ Service/            # Application services (business logic)
│   └─ EventSubscriber/    # Symfony event subscribers
```

```
└─ var/
└─ vendor/
```

🔑 Where things go

- **Controllers** → `src/Controller/` Define routes and handle requests. Same as Symfony.
- **Services** → `src/Service/` Business logic (e.g., product importer, asset transformer).
- **Repositories** → `src/Repository/` Custom queries for Data Objects or Doctrine entities.
- **Event Subscribers** → `src/EventSubscriber/` Listen to Symfony or Pimcore events (e.g., object save, asset update).
- **Configuration** → `config/services.yaml`, `config/routes.yaml`, or `config/packages/*`.

✨ Minimal Example

Controller

```php
<?php
// src/Controller/ProductController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Pimcore\Model\DataObject\Product;

class ProductController extends AbstractController
{
    #[Route('/products', name: 'product_list')]
    public function list(): Response
    {
        $products = new Product\Listing();
        $products->setLimit(10);

        return $this->json([
            'products' => array_map(fn($p) => [
                'id' => $p->getId(),
```

```
                'name' => $p->getName(),
                'price' => $p->getPrice(),
            ], iterator_to_array($products))
        ]);
    }
}
```

👉 This creates `/products` API that fetches Pimcore **Data Objects** of class `Product`.

Service

```
<?php
// src/Service/ProductService.php
namespace App\Service;

use Pimcore\Model\DataObject\Product;

class ProductService
{
    public function getFeaturedProducts(int $limit = 5): array
    {
        $listing = new Product\Listing();
        $listing->setLimit($limit);
        $listing->setOrderKey('price');
        $listing->setOrder('DESC');

        return $listing->load();
    }
}
```

---

services.yaml

```
# config/services.yaml
services:
    App\Service\ProductService:
        arguments: []
        public: true
```

Now you can inject it anywhere, e.g., in a controller:

```php
public function list(ProductService $productService): Response
{
    $products = $productService->getFeaturedProducts();
    return $this->json($products);
}
```

⚡ Common Integration Points

- **Routing** → Either via `#[Route]` attributes (preferred) or `config/routes.yaml`.
- **Dependency Injection** → Exactly Symfony style (`services.yaml` or PHP config).
- **Doctrine** → Standard repositories in `src/Repository/`, plus Pimcore's DataObject Listings.
- **Events** →
  - Symfony events (e.g., `kernel.request`).
  - Pimcore-specific (e.g., `pimcore.dataobject.preUpdate`).
- **Bundles** → You can package features as Pimcore bundles, just like Symfony bundles, under `/bundles`.

✅ In short: *Pimcore doesn't reinvent Symfony — it extends it.* You still structure controllers, services, and configs as in any Symfony project.

👉 Do you want me to show you a **Pimcore EventSubscriber** example (e.g., listening to a Product save event and automatically generating a SKU), or keep focusing on controllers & services?

08.md

Perfect — let's build a **REST API in Pimcore** to fetch `Product` objects as JSON, with **pagination** and **filtering**. Pimcore makes this smooth because it's just Symfony under

the hood.

🛠️ Step 1: Routing

You can use **PHP attributes** (preferred) or YAML.

Option A: Attribute Routing (modern way)

```php
// src/Controller/Api/ProductApiController.php
namespace App\Controller\Api;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\Routing\Annotation\Route;
use Pimcore\Model\DataObject\Product;

class ProductApiController extends AbstractController
{
    #[Route('/api/products', name: 'api_products', methods: ['GET'])]
    public function list(Request $request): JsonResponse
    {
        $page  = max(1, (int)$request->query->get('page', 1));
        $limit = min(50, (int)$request->query->get('limit', 10)); // cap for
safety
        $name  = $request->query->get('name'); // filter param

        $listing = new Product\Listing();
        $listing->setLimit($limit);
        $listing->setOffset(($page - 1) * $limit);

        if ($name) {
            $listing->addConditionParam("name LIKE ?", ["%$name%"]);
        }

        $items = [];
        foreach ($listing as $product) {
            $items[] = [
                'id'          => $product->getId(),
                'sku'         => $product->getSku(),
                'name'        => $product->getName(),
                'price'       => $product->getPrice(),
                'description' => $product->getDescription(),
            ];
        }

        return $this->json([
            'page'     => $page,
            'limit'    => $limit,
            'count'    => $listing->getTotalCount(),
            'products' => $items,
```

```
        ]);
    }
}
```

🛠️ Step 2: Serialization

Here we manually built the JSON. If you want more **Symfony-style serialization**, you can:

- Use Symfony Serializer component (`symfony/serializer`)
- Use `JMSSerializerBundle`

But for simple APIs, array-building is fastest.

🛠️ Step 3: Example curl Request

```
# Fetch first page, 5 products
curl "http://localhost/api/products?page=1&limit=5"

# Filter by name containing 'iPhone'
curl "http://localhost/api/products?page=1&limit=10&name=iPhone"
```

Sample JSON response


```
{
  "page": 1,
  "limit": 5,
  "count": 27,
  "products": [
    {
      "id": 123,
      "sku": "IP16-256GB",
      "name": "iPhone 16 Pro",
      "price": 119999,
      "description": "Latest flagship phone"
    },
    {
      "id": 124,
      "sku": "IP16-512GB",
      "name": "iPhone 16 Pro Max",
      "price": 139999,
      "description": "Bigger screen, more storage"
```

```
    }
  ]
}
```

⚡ Things to Note

- **Security** → For public APIs, configure API auth (e.g., Symfony Security, JWT).
- **Pagination** → Always cap `limit` (e.g., max 50) to avoid huge DB queries.
- **Filtering** → You can add more `addConditionParam` for `sku`, `price > ?`, etc.
- **Performance** → Use Elasticsearch integration for big catalogs (>100k products).

✅ You now have a working REST API for Products in Pimcore with pagination + filtering.

09.md

Great question 🙌 — Pimcore is built for **headless delivery**, but there are two main approaches depending on whether you want **GraphQL** or just REST.

1️⃣ Option A: Pimcore DataHub (GraphQL)

Pimcore doesn't ship with GraphQL "out of the box," but the **DataHub** bundle (official Pimcore extension) adds GraphQL and other APIs.

🔧 Installation

From your Pimcore project root:

```
composer require pimcore/data-hub
bin/console pimcore:bundle:enable PimcoreDataHubBundle
bin/console pimcore:bundle:install PimcoreDataHubBundle
```

This registers DataHub in your project.

🛠️ Create a GraphQL Endpoint

1. In the **Admin UI** → `Settings > DataHub`.
2. Click **Add Configuration**.
3. Select **GraphQL** as type.
4. Choose the **Data Objects Class** you want to expose (e.g., `Product`).
5. Configure which fields/relations are exposed.
6. Save → Pimcore automatically generates a GraphQL endpoint.

👉 Default endpoint is at:

```
/graphql/{configName}
```

For example:

```
http://localhost/graphql/products
```

🔍 Example GraphQL Query

```
query {
  getProductListing(first: 5) {
    edges {
      node {
        id
        name
        sku
        price
        description
        images {
          edges {
            node {
              path
              thumbnails {
                product_preview {
                  path
                }
```

```
            }
          }
        }
      }
      categories {
        edges {
          node {
            id
            name
          }
        }
      }
    }
  }
}
```

✅ This will return products with images (and their thumbnail variants) and categories.

2️⃣ Option B: Headless Without GraphQL

If you don't want DataHub (or your client doesn't allow it):

- Use **classic Symfony controllers** (REST endpoints, as we built earlier).
- Or use Pimcore's built-in **Web2Print / Output channels** for JSON/XML.
- Or expose Data Objects via **custom API bundles**.

Best alternative headless strategy:

- Build **REST endpoints** in Symfony controllers.
- Use Pimcore's **Assets Thumbnails** for optimized media URLs.
- Optionally add **API Platform** (a Symfony bundle) to auto-generate JSON:API endpoints.

⚡ Quick Comparison

Approach	Pros ✅	Cons ❌
DataHub (GraphQL)	Flexible queries, frontend devs love it, configurable in Admin UI	Extra bundle, some overhead

Approach	Pros ✅	Cons ❌
Custom REST (Symfony)	Full control, lightweight, no extra dependencies	Fixed responses, less frontend flexibility
API Platform	Auto-generated REST/GraphQL APIs, powerful filters	Extra setup, may feel heavy

👉 In production:

- If you want **frontend teams (React, Next.js, Vue, etc.) to query freely** → use **DataHub GraphQL**.
- If you want **strict APIs for performance/security** → use **custom REST endpoints**.

10.md

Perfect 👌 — Pimcore is Symfony-based, so **bundles = feature modules**. They package controllers, services, configs, and even admin UI code. This is how you'd ship reusable extensions (either private or to share with others).

Let's go step by step and build a **"DemoBundle"**.

🏗️ Step 1: Bundle Directory Layout

Inside your Pimcore project root (not inside `/src`), create a new folder:

```
bundles/
└─ DemoBundle/
   ├─ composer.json
   ├─ DemoBundle.php
   ├─ src/
   │  ├─ Controller/
   │  │  └─ DefaultController.php
   │  ├─ Service/
   │  │  └─ GreetingService.php
   │  └─ Resources/config/services.yaml
```

👉 Convention: custom bundles live in `/bundles/` so they are isolated from your app logic in `/src/`.

📝 Step 2: `composer.json` (for the bundle itself)

```
{
  "name": "mycompany/demo-bundle",
  "type": "pimcore-bundle",
  "description": "A demo Pimcore bundle",
  "autoload": {
    "psr-4": {
      "DemoBundle\\": "src/"
    }
  },
  "require": {
    "php": ">=8.1"
  }
}
```

This allows the bundle's namespace (`DemoBundle\`) to map to `/bundles/DemoBundle/src`.

📝 Step 3: The Bundle Class

```php
<?php
// bundles/DemoBundle/DemoBundle.php
namespace DemoBundle;

use Pimcore\Extension\Bundle\AbstractPimcoreBundle;

class DemoBundle extends AbstractPimcoreBundle
{
    public function getJsPaths(): array
    {
        // if you had custom admin UI JS files
        return [];
    }

    public function getCssPaths(): array
    {
        return [];
```

```
    }
}
```

This registers your bundle with Pimcore.

🛠️ Step 4: A Sample Service

```php
<?php
// bundles/DemoBundle/src/Service/GreetingService.php
namespace DemoBundle\Service;

class GreetingService
{
    public function greet(string $name): string
    {
        return "Hello, " . $name . "!";
    }
}
```

🛠️ Step 5: A Controller

```php
<?php
// bundles/DemoBundle/src/Controller/DefaultController.php
namespace DemoBundle\Controller;

use DemoBundle\Service\GreetingService;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class DefaultController extends AbstractController
{
    #[Route('/demo/hello/{name}', name: 'demo_hello')]
    public function hello(string $name, GreetingService $greeter): Response
    {
        return $this->json([
            'message' => $greeter->greet($name)
        ]);
    }
}
```

This will expose `/demo/hello/Sandeep` → `{ "message": "Hello, Sandeep!" }`.

📝 Step 6: services.yaml

```
# bundles/DemoBundle/src/Resources/config/services.yaml
services:
    DemoBundle\:
        resource: '../../*'
        exclude: '../../{Resources}'
```

This auto-registers your services.

⚡ Step 7: Enable the Bundle in Pimcore

Run:

```
bin/console pimcore:bundle:enable DemoBundle
bin/console pimcore:bundle:install DemoBundle
```

Check enabled bundles with:

```
bin/console debug:bundle
```

✅ Result

- You now have a Pimcore extension that ships with:
  - Its own **namespace** (`DemoBundle`)
  - A **service** (`GreetingService`)
  - A **controller endpoint** (`/demo/hello/{name}`)
- The bundle is **self-contained** → reusable across projects.

📦 **Pro tip**: If you plan to share the bundle publicly, push it to GitHub/Packagist and install it via Composer (`composer require mycompany/demo-bundle`).

11.md

Perfect 👌 — Pimcore builds on Symfony's **event system**, and adds its own events for **Data Objects, Assets, and Documents**. You can hook into them with **event subscribers** or **listeners**.

Let's create one that automatically updates a `slug` field whenever a `Product` object is saved.

🔔 Step 1: Choose the Right Event

For Data Objects Pimcore fires:

- `pimcore.dataobject.preUpdate` → before saving an existing object
- `pimcore.dataobject.postUpdate` → after saving
- `pimcore.dataobject.preAdd` → before creating a new object
- `pimcore.dataobject.postAdd` → after creating

👉 In our case, we want **preUpdate + preAdd** (so slug is set before persisting).

🛠️ Step 2: Event Subscriber Class

```php
<?php
// src/EventSubscriber/ProductSlugSubscriber.php
namespace App\EventSubscriber;

use Pimcore\Event\DataObjectEvents;
use Pimcore\Event\Model\DataObjectEvent;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Pimcore\Model\DataObject\Product;

class ProductSlugSubscriber implements EventSubscriberInterface
{
    public static function getSubscribedEvents(): array
```

```
    {
        return [
            DataObjectEvents::PRE_UPDATE => 'onProductSave',
            DataObjectEvents::PRE_ADD => 'onProductSave',
        ];
    }

    public function onProductSave(DataObjectEvent $event): void
    {
        $object = $event->getObject();

        if (!$object instanceof Product) {
            return;
        }

        $name = $object->getName();
        if ($name) {
            // simple slugify: lowercase + replace spaces
            $slug = strtolower(trim(preg_replace('/[^A-Za-z0-9-]+/', '-',
$name), '-'));
            $object->setSlug($slug);
        }
    }
}
```

📝 Step 3: Register the Subscriber

Add it in `config/services.yaml`:

```
services:
    App\EventSubscriber\ProductSlugSubscriber:
        tags:
            - { name: kernel.event_subscriber }
```

🛠️ Step 4: Ensure Your Class Has a `slug` Field

In your **Product class definition** (via Admin UI → Settings → Data Objects → Classes → Product):

- Add a field `slug` (type: *input*, read-only if you want editors not to touch it).

✅ What Happens

1. You save or create a `Product` object (`iPhone 16 Pro`).
2. Event fires → `onProductSave()` runs.
3. Slug field is auto-populated: `iphone-16-pro`.
4. Object is persisted with slug before commit.

⚡ Tips

- If you need **unique slugs**, you can query existing objects in `onProductSave` and append `-2`, `-3`, etc.
- For better slugging, use Symfony's **SluggerInterface** instead of regex.
- You can also use **postUpdate** if you need to act *after* save (e.g., syncing to external systems).

12.md

Great question 🚀 — CSV import is a **very common real-world Pimcore task**. You'll basically combine:

1. PHP CSV parsing (`fgetcsv` or `League\Csv`)
2. Pimcore Data Object API (`Pimcore\Model\DataObject\Product`)
3. Pimcore Asset API for images
4. Error handling & logging

Here's a **step-by-step PHP import script** you can run from a Symfony command or a standalone PHP script inside your Pimcore project.

🛠️ Example CSV

```
sku,name,price,description,image_url
P001,Phone X,999,Latest smartphone,https://example.com/images/phone.jpg
P002,Laptop Z,1499,High-end laptop,https://example.com/images/laptop.jpg
```

📄 Example Import Script

```php
<?php
// src/Command/ImportProductsCommand.php
namespace App\Command;

use Pimcore\Model\DataObject\Product;
use Pimcore\Model\Asset;
use Pimcore\Model\Asset\Image;
use Symfony\Component\Console\Command\Command;
use Symfony\Component\Console\Input\InputInterface;
use Symfony\Component\Console\Output\OutputInterface;
use Symfony\Component\Console\Style\SymfonyStyle;

class ImportProductsCommand extends Command
{
    protected static $defaultName = 'app:import-products';

    protected function execute(InputInterface $input, OutputInterface
$output): int
    {
        $io = new SymfonyStyle($input, $output);

        $csvPath = PIMCORE_PROJECT_ROOT . '/var/import/products.csv';
        if (!file_exists($csvPath)) {
            $io->error("CSV not found: $csvPath");
            return Command::FAILURE;
        }

        if (($handle = fopen($csvPath, 'r')) === false) {
            $io->error("Unable to open CSV file.");
            return Command::FAILURE;
        }

        $header = fgetcsv($handle, 1000, ','); // first row is header
        $count = 0;

        while (($row = fgetcsv($handle, 1000, ',')) !== false) {
            $data = array_combine($header, $row);

            try {
                $sku = $data['sku'] ?? null;
                if (!$sku) {
                    throw new \Exception("Missing SKU in row");
                }

                // 🔍 Find existing product or create new
                $product = Product::getBySku($sku, 1); // custom getter for
field "sku"
                if (!$product) {
                    $product = new Product();
```

```
                $product->setKey(\Pimcore\File::getValidFilename($sku));
                $product->setParentId(1); // put under "Home" (change as
needed)
                $product->setPublished(true);
                $product->setSku($sku);
            }

            $product->setName($data['name'] ?? '');
            $product->setPrice((float)($data['price'] ?? 0));
            $product->setDescription($data['description'] ?? '');

            // 🖼 Handle image
            if (!empty($data['image_url'])) {
                $asset = $this->downloadImage($data['image_url'], $sku);
                if ($asset instanceof Image) {
                    $product->setImages([$asset]); // assumes "images"
is a field (multihref or image)
                }
            }

            $product->save();
            $count++;
            $io->success("Imported/Updated product: {$sku}");
        } catch (\Throwable $e) {
            $io->error("Error importing row: " . $e->getMessage());
        }
    }

    fclose($handle);

    $io->success("Import finished. Total products: $count");

    return Command::SUCCESS;
}

private function downloadImage(string $url, string $sku): ?Image
{
    $contents = @file_get_contents($url);
    if ($contents === false) {
        throw new \Exception("Failed to download image from $url");
    }

    $filename = basename(parse_url($url, PHP_URL_PATH));
    $filename = $sku . '-' . $filename;

    // save under /imported
    $folder = Asset::getByPath('/imported');
    if (!$folder) {
        $folder = new Asset\Folder();
        $folder->setKey('imported');
        $folder->setParentId(1); // root
        $folder->save();
    }

    // check if image already exists
    $existing = Asset::getByPath('/imported/' . $filename);
    if ($existing instanceof Image) {
```

```
            return $existing;
        }

        $image = new Image();
        $image->setParent($folder);
        $image->setFilename($filename);
        $image->setData($contents);
        $image->save();

        return $image;
    }
}
```

🔧 Usage

1. Save this command as `src/Command/ImportProductsCommand.php`
2. Place `products.csv` in `var/import/`
3. Run:

```
bin/console app:import-products
```

4. Check in Pimcore Admin → Data Objects → Product

⚡ Error Logging

- Uses `SymfonyStyle` (`$io->error()`, `$io->success()`) → logs directly to CLI
- Wraps import in `try/catch` so a single bad row won't stop the whole process
- You could also log to `var/log/import.log` with `\Pimcore\Log\Simple::log()`

✅ **Result:**

- Products are **created or updated** by SKU
- Images are **downloaded + saved as Assets** under `/imported`
- Failures are **logged cleanly**

👉 Do you want me to extend this so it **handles product categories (object relations)** from the CSV too, or keep it simple for now?

13.md

Awesome — this is where Pimcore gets fun 🎉. Because Pimcore's **Admin UI is ExtJS-based**, you can extend it with your own menus, panels, and buttons. At a high level you need to:

1. Create a **Bundle** (you already know how).
2. Add a **custom controller + route** for your admin backend page.
3. Register a **custom JS module** so Pimcore loads it in the Admin UI.
4. Use that JS to add a **menu entry or button** to grids like Products.

🏗️ File Layout (bundle example)

```
bundles/
└─ DemoBundle/
   ├─ DemoBundle.php
   ├─ src/
   │  ├─ Controller/
   │  │  └─ AdminController.php
   │  └─ Resources/
   │     └─ public/
   │        └─ js/
   │           └─ admin.js
```

⚡ Step 1: Bundle Class (register JS)

```php
<?php
// bundles/DemoBundle/DemoBundle.php
namespace DemoBundle;

use Pimcore\Extension\Bundle\AbstractPimcoreBundle;

class DemoBundle extends AbstractPimcoreBundle
{
```

```
    public function getJsPaths(): array
    {
        return [
            '/bundles/demobundle/js/admin.js', // Pimcore auto-publishes
from Resources/public
        ];
    }
}
```

⚡ Step 2: Backend Controller

```
<?php
// bundles/DemoBundle/src/Controller/AdminController.php
namespace DemoBundle\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\Routing\Annotation\Route;

#[Route('/admin/demo', name: 'demo_admin_')]
class AdminController extends AbstractController
{
    #[Route('/hello', name: 'hello')]
    public function hello(): JsonResponse
    {
        return new JsonResponse(['message' => 'Hello from Demo Admin
Controller!']);
    }
}
```

👉 This exposes `/admin/demo/hello` for your JS to call.

⚡ Step 3: Custom Admin JS Module

```
// bundles/DemoBundle/src/Resources/public/js/admin.js

pimcore.registerNS("pimcore.plugin.DemoBundle");

pimcore.plugin.DemoBundle = Class.create(pimcore.plugin.admin, {
    getClassName: function () {
        return "pimcore.plugin.DemoBundle";
    },
```

```
    initialize: function () {
        pimcore.plugin.broker.registerPlugin(this);
    },

    // add a custom menu entry in Pimcore backend
    pimcoreReady: function (params, broker) {
        var toolbar = pimcore.globalmanager.get("layout_toolbar");

        toolbar.add({
            text: "Demo Page",
            iconCls: "pimcore_icon_overlay_add",
            handler: function () {
                Ext.Ajax.request({
                    url: "/admin/demo/hello",
                    success: function (response) {
                        var data = Ext.decode(response.responseText);
                        Ext.Msg.alert("Demo Bundle", data.message);
                    }
                });
            }
        });
    }
});

new pimcore.plugin.DemoBundle();
```

✅ This adds a new toolbar button “Demo Page” that shows an alert with server data.

⚡ Step 4: Add Button to Product Grid

To extend existing object grids (like Product list):

```
// inside admin.js, extend the object grid
pimcore.object.grid.prototype.getToolbar =
pimcore.object.grid.prototype.getToolbar.createSequence(function () {
    if (this.class === "Product") {
        this.toolbar.add({
            text: "Custom Action",
            iconCls: "pimcore_icon_apply",
            handler: function () {
                var selected = this.getSelectionModel().getSelection();
                if (selected.length) {
                    Ext.Msg.alert("Custom Action", "Selected product ID: " +
selected[0].data.id);
                }
            }.bind(this)
        });
    }
});
```

👉 This injects a **“Custom Action”** button into the Product grid toolbar.

⚡ How it fits together

- **PHP side**: your bundle’s `getJsPaths()` tells Pimcore to load `admin.js`.
- **Admin JS**: uses `pimcore.plugin.broker.registerPlugin()` to hook into Pimcore UI lifecycle.
- **Custom Controller**: handles AJAX calls for your custom page/button.
- **Result**: editors see new UI buttons and panels seamlessly integrated.

✅ Minimal Workflow Recap

1. Create bundle → register JS with `getJsPaths()`
2. Add `admin.js` → extend UI (toolbar, grid buttons, panels)
3. Add backend controller → provide JSON APIs for UI
4. Run `bin/console pimcore:bundle:enable DemoBundle`

Would you like me to also show you how to make a **full custom Admin Panel tab (like a React-style view in Pimcore’s ExtJS UI)**, or keep it to buttons + menus?

14.md

Excellent topic ⚡ — Pimcore is powerful but can get heavy if not tuned right. Performance is usually about **cache layers, database tuning, search indexing, and profiling**. Let’s break it down.

🚀 1. Cache Layers

✅ PHP OPcache

- Compiles PHP bytecode and keeps it in memory.
- Always enable in production.
- Config basics (`php.ini`):

```
opcache.enable=1
opcache.memory_consumption=256
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0  ; only for prod
```

✅ Pimcore Cache (Symfony Cache)

- Pimcore uses Symfony Cache for metadata, class definitions, and object bricks.
- Recommended: use **Redis** or **Memcached** instead of filesystem.
- Config (`config/packages/cache.yaml`):

```
framework:
  cache:
    app: cache.adapter.redis
    default_redis_provider: "redis://localhost"
```

✅ HTTP Reverse Proxy (Varnish / Symfony HttpCache)

- Full-page caching for CMS pages and APIs.
- Pimcore can send cache headers (`Cache-Control`, `ETag`).
- With Varnish/Nginx cache, most requests never hit PHP at all.

⚡ 2. Database Tuning (MySQL/MariaDB)

- **Schema design**: avoid too many nested relations in a single object.

- **Indexes**: add indexes on frequently filtered fields (`sku`, `slug`, etc.).
- **Buffer pool**:

```
innodb_buffer_pool_size = 2G  ; ~60–70% of server RAM
```

- Avoid `SELECT *` queries in custom code.
- Use **lazy loading** (don't load images, relations unless needed).

🔎 3. Search Indexing

Built-in Fulltext Search

- Pimcore can use MySQL fulltext, but not great for large catalogs.

Elasticsearch / OpenSearch

- Use Pimcore DataHub or custom sync to ES.
- Best for: product catalogs, filtering, faceting.
- Flow: Pimcore saves → event hook → push/update document in ES.
- Example ES document:

```
{
  "sku": "P001",
  "name": "Phone X",
  "price": 999,
  "categories": ["Smartphones", "Electronics"]
}
```

🔧 4. Profiling a Slow Request

1. **Enable Symfony Profiler (dev env)**

- Shows database queries, execution time, cache hits.
- Run the request, then check `/app_dev.php/_profiler`.

2. **Use Blackfire.io or Tideways**
   - Flamegraphs for PHP execution (see bottlenecks).
3. **Check Pimcore Logs** (`var/log/dev.log` / `prod.log`)
   - Look for slow query warnings.
4. **MySQL slow query log**

```
slow_query_log = 1
long_query_time = 1
```

→ Helps you find bad queries Pimcore or custom code runs.

🧩 5. Quick Checklist

- ✅ Enable OPcache + Redis cache
- ✅ Configure reverse proxy (Varnish/Nginx fastcgi_cache)
- ✅ Optimize MySQL (buffer pool, indexes, slow query log)
- ✅ Use Elasticsearch for large catalogs / faceted search
- ✅ Profile with Symfony Profiler + Blackfire
- ✅ Reduce unnecessary relations/field collections in models

15.md

Great topic 🔐 — Pimcore has a **granular user & permission model** because editors, developers, and API consumers all interact differently. Let's break it down into **users, roles, object/document/asset permissions, API security, and auditing**.

👤 1. Users & Roles

- **Users** = actual accounts (human or system).
- **Roles** = reusable permission sets that can be assigned to multiple users.
- Users can belong to multiple roles; permissions are merged.
- Define roles first (e.g. *Content Editor*, *Product Manager*, *API Client*) and then add users.

👉 Admin UI → *Settings* > *Users / Roles*.

🗂️ 2. Object / Document / Asset Permissions

Pimcore's tree-based structure makes it natural to secure content hierarchically.

Permissions include:

- **Read / Write / Delete / Publish** (per type)
- **List / View / Create children**
- **Allow variants** (objects)

Best practices:

- Put related content under folders (e.g. `/Products`, `/MarketingDocs`).
- Apply permissions at folder root → children inherit.
- Use **Roles** to grant per-folder access (e.g. *Product Manager* can edit `/Products/*` but not `/Customers`).
- Keep the **Admin** role only for superusers; give editors minimal access.

🌐 3. Securing APIs

REST / Symfony Controllers

- Use Symfony **security.yaml**:

```
security:
  firewalls:
    api:
      pattern: ^/api
      stateless: true
      json_login:
        check_path: /api/login
  access_control:
    - { path: ^/api, roles: ROLE_API_USER }
```

Pimcore DataHub (GraphQL)

- Assign permissions per **endpoint**.
- Example: Product API may allow only `query` (no `mutation`).
- Control which fields are exposed via DataHub config.

Best practices:

- Use **stateless JWT or API keys** for machine-to-machine access.
- Never expose the admin UI endpoints (`/admin/*`) to the public internet.
- Rate-limit APIs at the proxy/load balancer level.

📝 4. Auditing Admin Actions

- Pimcore logs user actions in `var/log/admin-*.log`. Example:

```
2025-09-26 12:01:23 User "editor1" published object ID 123
```

- You can also enable **Versioning** for Data Objects → history of changes is stored.
- Use **Activity Store** (Enterprise edition) or write a custom **EventSubscriber** for `pimcore.admin.events` to push changes into an audit log.

👉 Example: log when a Product is deleted:

```
use Pimcore\Event\AdminEvents;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
```

```php
use Symfony\Component\HttpFoundation\Request;

class AdminAuditSubscriber implements EventSubscriberInterface
{
    public static function getSubscribedEvents(): array
    {
        return [
            AdminEvents::ADMIN_LOGIN => 'onAdminLogin',
        ];
    }

    public function onAdminLogin($event): void
    {
        /** @var Request $request */
        $request = $event->getRequest();
        $user = $event->getUser();
        \Pimcore\Log\Simple::log('admin_audit', "User {$user->getName()}
logged in from {$request->getClientIp()}");
    }
}
```

✅ Best Practices Recap

- Create **roles** instead of managing permissions per user.
- Secure **folders** (Products, Media, Content) instead of single objects.
- Restrict **API access** with firewalls, roles, and DataHub configs.
- **Audit admin actions**: enable logs + versioning + optional event subscribers.
- Keep **super-admins minimal**; always use least privilege.
- Separate **editor accounts** from **API/system accounts**.

16.md

Great debugging question 🛠️ — Pimcore is Symfony under the hood, so you use **Symfony tools + Pimcore-specific logs**. Let's break it into logs, profiler, SQL debugging, and common errors.

📜 1. Logs in Pimcore

Pimcore writes logs into `var/log/` (per environment).

- **dev.log / prod.log** → Symfony/Pimcore runtime logs
- **system.log** → Pimcore core system events
- **admin-YYYY-MM-DD.log** → backend actions (who saved/published what)
- **php_error.log** (depending on PHP setup)

Enable detailed logs

In `config/packages/monolog.yaml`:

```
monolog:
  handlers:
    main:
      type: stream
      path: "%kernel.logs_dir%/%kernel.environment%.log"
      level: debug   # use info in prod, debug in dev
```

👉 Command to tail logs:

```
tail -f var/log/dev.log
```

🧩 2. Symfony Profiler in Pimcore

Profiler is available in **dev mode** only.

- URL: `https://localhost/app_dev.php/_profiler`
- Or click Symfony **debug toolbar** at bottom of page.

You can see:

- Timeline of request (controller, Twig, events)
- SQL queries (with time + EXPLAIN link)
- Cache hits/misses
- Logs emitted during the request

👉 If toolbar is missing, ensure in `config/packages/dev/web_profiler.yaml`:

```
web_profiler:
  toolbar: true
  intercept_redirects: false
```

🗄️ 3. Debugging SQL Queries

Pimcore uses Doctrine ORM + its own query layers.

Show Doctrine queries

Add to `.env.local`:

```
APP_DEBUG=1
```

Or enable SQL logger in `doctrine.yaml`:

```
doctrine:
  dbal:
    logging: true
```

👉 Command to run SQL profiler manually:

```
bin/console doctrine:query:sql "SELECT * FROM object_query_1 WHERE
o_key='ProductX'"
```

Slow queries

Enable MySQL slow query log in `my.cnf`:

```
slow_query_log = 1
long_query_time = 1
```

🔒 4. Debugging Permissions

- Check **roles & users** in *Settings* > *Users*.
- If an editor cannot see objects: check permissions on the **folder root** (permissions don't cascade automatically if "list children" is unchecked).
- For APIs: check `security.yaml` firewalls → missing roles often block access silently.
- Use CLI to see user details:

```
bin/console pimcore:user:list
```

🚀 5. Debugging Deployments

Common pitfalls:

- **Class definition mismatch**
  - After deploying, always run:

```
bin/console pimcore:deployment:classes-rebuild -c -d
```

- **Cache issues**
  - Clear Pimcore/Symfony caches:

```
bin/console cache:clear
bin/console pimcore:cache:clear
```

- **File permissions**
  - `var/`, `public/var/`, and `web/var/` must be writable by the web server.
  - Fix:

```
chown -R www-data:www-data var/ public/var/
```

- **Missing migrations**
  - Run after updates:

```
bin/console doctrine:migrations:migrate
```

🛠️ 6. Extra Debug Commands

- List routes:

```
bin/console debug:router
```

- Show all registered services:

```
bin/console debug:container
```

- Check events being dispatched:

```
bin/console debug:event-dispatcher
```

- Test API endpoint:

```
curl -v http://localhost/api/products
```

✅ Best Practices

- Use **dev environment** locally (`APP_ENV=dev`, `APP_DEBUG=1`).
- Tail logs (`tail -f var/log/*.log`) while reproducing the issue.

- Check Symfony **profiler** for slow queries & cache misses.
- Use **MySQL slow query log** for DB bottlenecks.
- Always check **file permissions + cache clearing** after deployment errors.

👉 Do you want me to also prepare a **"debugging checklist" cheat sheet** (step-by-step sequence you can follow whenever Pimcore misbehaves), or keep it as these topic blocks?

17.md

Perfect question ✅ — Pimcore is Symfony under the hood, so **testing = PHPUnit + Symfony Test tools**, but you also need to deal with Pimcore bootstrapping. Let's go step by step.

⚙️ 1. PHPUnit Setup

Install PHPUnit (if not already)

```
composer require --dev phpunit/phpunit symfony/phpunit-bridge
```

`phpunit.xml.dist` (at project root)

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit
    bootstrap="vendor/autoload.php"
    colors="true"
    cacheResultFile=".phpunit.result.cache"
    failOnWarning="true"
    failOnRisky="true"
    stopOnFailure="false"
    verbose="true"
>
    <testsuites>
        <testsuite name="Application Test Suite">
            <directory>tests</directory>
        </testsuite>
```

```
    </testsuites>

    <php>
        <env name="APP_ENV" value="test"/>
        <env name="APP_DEBUG" value="1"/>
        <env name="PIMCORE_ENVIRONMENT" value="test"/>
    </php>
</phpunit>
```

👉 This ensures Pimcore/Symfony runs in a `test` **environment**.

🛠️ 2. Example Service & Test

Service (`src/Service/GreetingService.php`)

```php
<?php
namespace App\Service;

class GreetingService
{
    public function greet(string $name): string
    {
        return "Hello, " . ucfirst($name) . "!";
    }
}
```

Unit Test (`tests/Service/GreetingServiceTest.php`)

```php
<?php
namespace App\Tests\Service;

use App\Service\GreetingService;
use PHPUnit\Framework\TestCase;

class GreetingServiceTest extends TestCase
{
    public function testGreetReturnsFormattedMessage(): void
```

```
    {
        $service = new GreetingService();
        $result = $service->greet('sandeep');

        $this->assertSame("Hello, Sandeep!", $result);
    }
}
```

Run:

```
bin/phpunit
```

🖥️ 3. Functional Test (Controller)

Suppose you have a controller:

```
// src/Controller/Api/ProductController.php
namespace App\Controller\Api;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/api/test', name: 'api_test')]
    public function test(): JsonResponse
    {
        return $this->json(['message' => 'API works']);
    }
}
```

Test (`tests/Controller/ProductControllerTest.php`)

```
<?php
namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;
```

```php
class ProductControllerTest extends WebTestCase
{
    public function testApiEndpointReturnsJson(): void
    {
        $client = static::createClient();
        $client->request('GET', '/api/test');

        $this->assertResponseIsSuccessful();
        $this->assertResponseHeaderSame('content-type', 'application/json');
        $this->assertJsonStringEqualsJsonString(
            json_encode(['message' => 'API works']),
            $client->getResponse()->getContent()
        );
    }
}
```

🧪 4. Running Tests Locally

Run all tests:

```
bin/phpunit
```

Run a single test:

```
bin/phpunit tests/Service/GreetingServiceTest.php
```

Run with coverage:

```
XDEBUG_MODE=coverage bin/phpunit --coverage-html var/coverage
```

→ Open `var/coverage/index.html` in your browser.

⚡ 5. CI/CD Integration (GitHub Actions example)

`.github/workflows/tests.yml`

```yaml
name: Pimcore Tests

on: [push, pull_request]

jobs:
  tests:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8
        env:
          MYSQL_DATABASE: pimcore_test
          MYSQL_USER: root
          MYSQL_ALLOW_EMPTY_PASSWORD: yes
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping --silent"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=3
    steps:
      - uses: actions/checkout@v3
      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.1'
          extensions: mbstring, intl, pdo_mysql
          coverage: xdebug
      - name: Install dependencies
        run: composer install --prefer-dist --no-progress
      - name: Run tests
        run: bin/phpunit --testdox
```

✅ Best Practices

- Write **unit tests** for services (fast, no DB).
- Write **functional tests** for controllers/APIs (via Symfony's test client).
- Keep Pimcore-specific code (DataObjects, Assets) in **integration tests** — those need DB + Pimcore bootstrap.
- Use a separate **test database** (`pimcore_test`) and reset it before runs.
- Run tests in CI (GitHub Actions, GitLab CI, Jenkins).

👉 Do you want me to also show an **integration test example with Pimcore Data Objects** (e.g., create a Product in the DB, fetch it via API), or keep it at service +

controller level?

18.md

Great one 🙌 — this is where many Pimcore projects get messy. Let's lay out a **safe workflow for configuration + deployment**, tailored for Pimcore's Symfony foundation.

🔑 1. Environment Variables

- Store environment-specific values in `.env.local` (never commit to Git).
- Define defaults in `.env` but override via CI/CD or server config.
- Examples:

```
APP_ENV=prod
APP_DEBUG=0
DATABASE_URL="mysql://user:pass@mysql:3306/pimcore_prod"
PIMCORE_ENVIRONMENT=prod
PIMCORE_IMAGE_PROXY_SECRET=xxxx
```

👉 In Docker/Kubernetes: inject env vars through secrets, not `.env` files.

🗃️ 2. Database Changes

Pimcore stores data structure in **Class Definitions** (JSON in `var/classes`).

- Always commit those JSON files to Git.
- Use `bin/console pimcore:deployment:classes-rebuild` to sync DB schema.
- For custom tables/entities, use **Doctrine migrations**:

```
bin/console make:migration
bin/console doctrine:migrations:migrate --env=prod
```

👉 Rule: No manual DB changes! Everything via migrations or class definitions.

🖼️ 3. Assets (Files, Images, Docs)

- Assets live in `/var/assets` (or external storage).
- In CI/CD: don't overwrite! Use shared storage (e.g. NFS, S3, Azure Blob).
- For large deployments: serve assets via CDN.

🏗️ 4. Build & Cache

Steps for deploying to **staging/prod**:

1. `composer install --no-dev --optimize-autoloader`
2. `bin/console pimcore:deployment:classes-rebuild -c -d` (sync class defs + DB)
3. `bin/console doctrine:migrations:migrate --no-interaction --env=prod`
4. `bin/console cache:clear --env=prod`
5. Warmup caches:

```
bin/console cache:warmup --env=prod
bin/console pimcore:cache:clear --all
```

👉 Optional: run `bin/console assets:install public --symlink` if you added custom bundles.

🔁 5. Git/CI Workflow

Branching Strategy (simple but safe):

- `main` → production
- `develop` → staging
- feature branches → merge into `develop` → then release/merge to `main`

CI/CD Pipeline (example):

1. **On push to `develop`:**
    - Run tests (`bin/phpunit`).
    - Deploy to **staging server**.
    - Run migrations + class sync.
    - Notify QA team.
2. **On tag/release (`v1.2.0`) in Git:**
    - Deploy to **production**.
    - Same steps as staging (composer install, migrate, class sync, clear cache).
    - Run health checks (Symfony profiler, status page).

🛡️ 6. Extra Safety

- **Backups**: Always back up DB + assets before migration.
- **Review migrations** before running in prod.
- Use **feature flags** (env vars or DB toggle) for risky features.
- **Audit logs**: Pimcore already logs user/admin actions — keep them enabled.

✅ Workflow in One Sentence

👉 *Commit code + class defs → push → CI runs tests → deploys to staging → review → merge/release → deploy to production with migrations + cache rebuild, keeping assets in shared storage.*

19.md

Awesome goal 🎉 — contributing to **Pimcore core** will level up your skills and help the community. Let's walk through the whole contribution cycle.

🔍 1. Find "Good First Issues"

- Pimcore GitHub: https://github.com/pimcore/pimcore
- Go to **Issues** → **Labels** → `good first issue` or `help wanted`. 👉 Direct link: Good first issues
- Pick one that is small (typo fix, docs, small bug, minor feature).

🍴 2. Fork the Repo

```
# Clone your fork
git clone https://github.com/<your-username>/pimcore.git
cd pimcore

# Add upstream for syncing
git remote add upstream https://github.com/pimcore/pimcore.git
```

⚙️ 3. Core Development Environment

Pimcore uses Symfony + Composer + PHPUnit. To set up dev:

```
# Install deps
composer install

# Copy test config
cp .env.test.dist .env.test

# Create DB for tests (MySQL or MariaDB)
mysql -u root -p -e "CREATE DATABASE pimcore_test;"

# Run DB schema init
bin/console doctrine:migrations:migrate --env=test --no-interaction

# Run full test suite
vendor/bin/phpunit
```

👉 Some contributors use **Docker (pimcore/pimcore:PHP-8.x)** — but for core dev, local composer + phpunit is usually enough.

🛠️ 4. Make a Change

1. Create a branch from `10.x` (current main dev branch):

```
git checkout -b fix/typo-in-docblock
```

2. Apply your changes (fix a bug, improve a service, update docs).

3. Run tests:

```
vendor/bin/phpunit
bin/console lint:container
bin/console lint:twig
```

4. Commit with a clear message:

```
git commit -m "Fix typo in ProductService docblock"
```

🚀 5. Submit a PR

```
git push origin fix/typo-in-docblock
```

Go to your fork → GitHub suggests making a PR.

- Base branch: `pimcore/pimcore:10.x`
- Compare branch: your `fix/...`

📝 6. PR Description (Template)

Follow Pimcore's Contribution Guidelines.

Example PR body:

```
## Changes in this PR
- Fixed a typo in ProductService docblock.
- Added missing test case for product slug generation.

## Why this is needed
Improves code readability and prevents confusion for new developers.

## Related Issues
Closes #12345

## Checklist
- [x] Tests pass locally (`vendor/bin/phpunit`)
- [x] No BC breaks
- [x] Code follows Pimcore coding style
```

🧭 7. After PR

- The CI (GitHub Actions) will run — fix any failing checks.
- Core maintainers may request changes → update your branch → push again.
- Once approved → merged 🚀

🛡️ Best Practices

- Always branch from `10.x` (not `main`).
- Keep commits small & focused.
- Use **PSR-12 coding style** (`composer cs-fix` can help).
- Sync with upstream often:

  ```
  git fetch upstream
  git rebase upstream/10.x
  ```

✅ With this workflow you'll be able to confidently contribute to Pimcore's core.

👉 Do you want me to draft a **step-by-step "first PR exercise"** (like fixing a small typo in docs) so you can try contributing today with zero risk?

20.md

Perfect 🚀 — you're asking for a **structured learning roadmap** with concrete outputs. Let's design a **5-week, hands-on Pimcore learning plan** that moves from zero → a real open-source contribution.

📅 5-Week Pimcore Learning & Contribution Plan

Week 1: Install & Basics

🎯 *Goal: Have Pimcore running locally, know your way around the Admin UI, and commit first project code.*

Daily Tasks

- **Day 1–2:**
  - Install Pimcore with Docker + Composer.
  - Access Admin UI at `/admin`.
  - Explore *Data Objects, Assets, Documents* sections.
- **Day 3–4:**
  - Create a simple **Product** class (fields: name, price, sku).
  - Add a few objects manually.
- **Day 5:**
  - Explore **Assets** (upload images, PDFs).
  - Try linking assets to your product.

- **Day 6–7:**
  - Play with **Documents** (create a CMS page, add text/images).
  - Read Pimcore docs on Data Objects & [Assets].

✅ **Success Criteria:**

- Pimcore runs locally.
- Can create/update a Product object with an image in Admin UI.
- First commit: project bootstrapped in Git.

Week 2: Classes & Assets

🎯 *Goal: Model richer data and handle media programmatically.*

Daily Tasks

- **Day 1–2:**
  - Add fields: description, categories (relation), images (asset relation).
  - Use localized fields for multi-language.
- **Day 3:**
  - Write a PHP script to create/update Product objects programmatically.
- **Day 4:**
  - Import 5 products from a small CSV (use `fgetcsv`).
- **Day 5:**
  - Experiment with **thumbnails**: define a thumbnail preset in Admin UI.
- **Day 6:**
  - Create a thumbnail variant *programmatically*.
- **Day 7:**
  - Document your Product data model (diagram + notes).

✅ **Success Criteria:**

- Product model with relations + localized fields exists.
- CSV import script works.
- Thumbnail variant shows correctly for images.

Week 3: APIs & Import/Export

🎯 *Goal: Expose and consume Pimcore data via APIs.*

Daily Tasks

- **Day 1–2:**
  - Create a Symfony controller that returns Products as JSON.
  - Add pagination + filtering (`?page=2&category=shoes`).
- **Day 3:**
  - Test API with `curl` and Postman.
- **Day 4–5:**
  - Enable Pimcore **DataHub/GraphQL** bundle.
  - Write a GraphQL query to fetch products with images/categories.
- **Day 6:**
  - Write an export script: Products → CSV.
- **Day 7:**
  - Combine: Fetch via API → save into CSV → re-import.

✅ **Success Criteria:**

- REST endpoint works with pagination & filters.
- GraphQL query returns product + image data.
- Export + re-import cycle proven.

Week 4: Admin Extensions & Testing

🎯 *Goal: Extend Admin UI and test Pimcore code.*

Daily Tasks

- **Day 1–2:**
  - Add a **custom button** in Product grid (e.g. “Export CSV”).
- **Day 3–4:**
  - Build a small **bundle** with a custom admin page (e.g. product stats).
- **Day 5:**
  - Set up PHPUnit config.
  - Write a **unit test** for a simple service.
- **Day 6–7:**
  - Write a **functional test** for your Product API endpoint.

✅ **Success Criteria:**

- Custom button appears in Product grid.
- Bundle loads a custom admin page.
- PHPUnit runs with passing unit + functional test.

Week 5: Contribution & PR

🎯 *Goal: Submit your first Pimcore PR.*

Daily Tasks

- **Day 1:**
  - Read Pimcore’s Contributing Guidelines.
- **Day 2–3:**

    - Find a “good first issue” on GitHub issues.
    - Fork Pimcore repo, set up local test env.

- **Day 4–5:**
    - Fix the issue (bug, typo, docs, small feature).
    - Run full test suite (`vendor/bin/phpunit`).

- **Day 6:**
    - Commit → Push → Open PR against Pimcore’s `10.x` branch.
    - Follow PR template (describe changes, link issue, checklist).

- **Day 7:**
    - Respond to maintainer feedback if any.

✅ **Success Criteria:**

- You have an open PR in Pimcore core repo.
- All tests pass in CI.
- You’ve contributed 🎉

📌 Summary Timeline

- **Week 1:** Setup & basics → Product object via Admin UI.
- **Week 2:** Data modeling & Assets → CSV import + thumbnails.
- **Week 3:** APIs → REST + GraphQL + Import/Export.
- **Week 4:** Admin Extension + Tests → Custom UI + PHPUnit.
- **Week 5:** Core Contribution → Fork, fix, PR submitted.

👉 Do you want me to turn this into a **checklist format (markdown table)** so you can tick things off daily as you progress?

---

importent/01.md

Got it! You want to **create a custom controller in a Pimcore package** and have it respond to routes. I’ll walk you **step by step** for a clean setup.

1️⃣ Create a custom package

If you haven't already, generate a package. Let's assume your package is called `MyPackage`. The folder structure should look like this:

```
/src/MyPackage/
├── Controller/
│   └── AdminController.php
├── Resources/
│   └── config/
│       └── routes.yaml
├── MyPackageBundle.php
```

2️⃣ Create the Bundle class

```php
// src/MyPackage/MyPackageBundle.php
namespace App\MyPackage;

use Symfony\Component\HttpKernel\Bundle\Bundle;

class MyPackageBundle extends Bundle
{
}
```

3️⃣ Create the Controller

```php
// src/MyPackage/Controller/AdminController.php
namespace App\MyPackage\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class AdminController extends AbstractController
{
    #[Route('/my-package/hello', name: 'my_package_hello')]
    public function hello(): Response
```

```
    {
        return new Response('Hello from MyPackage!');
    }
}
```

✅ Notes:

- The namespace must match the folder path.
- You can use **Symfony route annotations** (`#[Route(...)]`) or YAML routes.

4️⃣ Register routes (YAML alternative)

If you prefer YAML:

```
# src/MyPackage/Resources/config/routes.yaml
my_package_hello:
    path: /my-package/hello
    controller: App\MyPackage\Controller\AdminController::hello
```

Then import it in `config/routes.yaml`:

```
my_package:
    resource: '../src/MyPackage/Resources/config/routes.yaml'
```

5️⃣ Ensure controller is registered as service

Check `config/services.yaml`:

```
services:
    App\MyPackage\:
        resource: '../src/MyPackage/*'
        exclude:
'../src/MyPackage/{DependencyInjection,Entity,Migrations,Tests}'
        tags: ['controller.service_arguments']
```

This will make your controllers **callable by Symfony / Pimcore**.

6️⃣ Clear cache

Pimcore heavily caches routes and services. Run:

```
bin/console cache:clear
```

7️⃣ Test

Open your browser:

```
https://your-pimcore-domain/my-package/hello
```

You should see:

```
Hello from MyPackage!
```

If you want, I can also show how to **add an admin-only route** inside your Pimcore backend so it **appears under the admin panel menu**. That way, your controller is only accessible to authenticated Pimcore users.

Do you want me to show that?